รายงานโครงงานการวิเคราะห์และออกแบบระบบ (Project Report)

ระบบบริหารจัดการร้าน Minions Cafe (Coffee Shop Management & POS System)

กลุ่มที่ 14: Minions Cafe

รายวิชา 88734065 การวิเคราะห์และออกแบบระบบ (System Analysis and Design)
คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา

สมาชิกในกลุ่ม:

1. **67160002** นางสาวชาลิสา ศรีหงษ์
2. **67160042** นางสาวมนทกานต์ ปาลี
3. **67160178** นางสาวจตุพร ธรรมฤทธิ์
4. **67160190** นางสาวบุษชยาภา คะสุดใจ
5. **67160292** นางสาวชนิดาภา ชาภักดี

สารบัญ (Table of Contents)

1. บทนำ (Introduction)
2. การวิเคราะห์ความเป็นไปได้ (Feasibility Study)
3. การวิเคราะห์ความต้องการ (Requirements Analysis)
4. User Stories และ Use Case Diagram
5. การออกแบบเชิงวัตถุ (Object-Oriented Analysis & Design - OOAD)
6. การออกแบบฐานข้อมูล (Database Design)
7. การออกแบบกระบวนการและพฤติกรรมระบบ (Process & Behavioral Design)
8. การออกแบบ Input/Output และส่วนประสานผู้ใช้ (UI/UX & Wireframe Design)
9. สถาปัตยกรรมระบบ (System Architecture - 4+1 View Model)
10. ผลการทดสอบ Usability (Usability Testing Results)
11. ผลการรวมระบบ (System Integration & End-to-End Testing)
12. ผลการตรวจสอบคุณภาพและความปลอดภัยของซอฟต์แวร์ (Code Quality & Security Review)
13. สรุปผลและข้อเสนอแนะ (Conclusions & Recommendations)
14. ภาคผนวก (Appendix: AI Disclosure Forms)

---

1. บทนำ (Introduction)

1.1 บริบทและความเป็นมาของร้านกาแฟ (Background)

ร้านกาแฟ Minions Cafe เป็นธุรกิจร้านกาแฟสดและเครื่องดื่มที่มีลูกค้าเข้ามาใช้บริการอย่างต่อเนื่อง และมีแนวคิดในการขยายการดำเนินงานแบบหลายสาขา (Multi-branch) เดิมร้านประสบปัญหาจากการจดออเดอร์ด้วยกระดาษ ซึ่งมักเกิดความผิดพลาดในการส่งต่อรายการให้บาริสต้า ความล่าช้าในการคำนวณราคารวมและเงินทอนในช่วงเวลาเร่งด่วน การควบคุมสต็อกวัตถุดิบที่มักขาดสต็อกโดยไม่รู้ตัวล่วงหน้า และเจ้าของร้านไม่สามารถดูยอดขายสรุปได้ทันที ต้องรอรวบรวมใบเสร็จเมื่อปิดร้าน

1.2 วัตถุประสงค์ของโครงงาน (Objectives)

1. เพื่อวิเคราะห์และพัฒนาระบบขายหน้าร้าน (Point of Sale - POS) ที่ช่วยบันทึกออเดอร์ คำนวณราคาอัตโนมัติ และออกใบเสร็จรับเงินได้อย่างถูกต้องรวดเร็ว
2. เพื่อพัฒนาระบบตัดสต็อกวัตถุดิบและสินค้าแบบอัตโนมัติ พร้อมแจ้งเตือนเมื่อสต็อกสินค้าต่ำกว่าเกณฑ์
3. เพื่อพัฒนาระบบรายงานสรุปยอดขายแบบเรียลไทม์ (Real-time Sales Report) ให้เจ้าของร้านดูยอดขายรวมและเมนูขายดีแยกรายสาขาได้ทันที
4. เพื่อศึกษาและประยุกต์ใช้วงจรการพัฒนาระบบแบบ Agile (Iterative & Incremental) ตั้งแต่การวิเคราะห์ความต้องการ การออกแบบ UML และการทดสอบกับผู้ใช้จริง

---

2. การวิเคราะห์ความเป็นไปได้ (Feasibility Study)

2.1 การศึกษาความเป็นไปได้ตามกรอบ TELOS

- **Technical Feasibility:** มีความเป็นไปได้สูง ทีมพัฒนามีความรู้ความสามารถในการพัฒนาเว็บแอปพลิเคชันด้วย HTML/CSS/JavaScript, Node.js/Express, และฐานข้อมูล MySQL 8.0
- **Economic Feasibility:** คุ้มค่าต่อการลงทุน ช่วยลดต้นทุนความผิดพลาดจากกระดาษและมนุษย์ ลดเวลาการรอคอยของลูกค้า เพิ่มยอดขายและผลกำไรในระยะยาว
- **Operational Feasibility:** พนักงานร้านและเจ้าของร้านสามารถเรียนรู้ระบบได้รวดเร็ว เนื่องจากออกแบบส่วนประสานผู้ใช้ให้เรียบง่าย (Intuitive & User-friendly) ตรงกับงานประจำวัน
- **Schedule Feasibility:** สามารถส่งมอบระบบงานและเอกสารได้ครบถ้วนภายใน 1 ภาคการศึกษา (15 สัปดาห์) ตามแผนงาน

2.2 การเปรียบเทียบกระบวนการทำงานแบบเดิม (As-Is) และแบบใหม่ (To-Be)

- **กระบวนการเดิม (As-Is):** พนักงานจดรายการลงสมุด → ยื่นให้บาริสต้าชง → คิดเงินด้วยเครื่องคิดเลข → ลูกค้าจ่ายเงิน → พนักงานจดลงสมุดรายรับรายวัน (เกิดคอขวดและผิดพลาดบ่อย)
- **กระบวนการใหม่ (To-Be):** พนักงานกดเลือกเมนูผ่านจอ POS → ระบบคำนวณเงินและแสดงยอดรวมทันที → รับชำระเงินและพิมพ์ใบเสร็จ → ระบบตัดสต็อกอัตโนมัติ → ข้อมูลยอดขายส่งตรงเข้า Dashboard รายงานของเจ้าของร้านทันที

---

3. การวิเคราะห์ความต้องการ (Requirements Analysis)

3.1 บันทึกการสัมภาษณ์และปัญหาที่พบ (User Interviews)

จากการสัมภาษณ์เจ้าของร้านและพนักงาน พบประเด็นสำคัญ:

- ออเดอร์ตกหล่นและคิดเงินผิดเฉลี่ย 3-5 รายการต่อวันในช่วงลูกค้าหนาแน่น
- วัตถุดิบนมสดและเมล็ดกาแฟมักหมดกะทันหัน ทำให้เสียโอกาสในการขาย
- เจ้าของร้านต้องการทราบว่าแต่ละวันเมนูใดขายดีที่สุดเพื่อวางแผนสต็อกล่วงหน้า

3.2 ข้อกำหนดความต้องการ (Requirements Summary)

- **Functional Requirements:**
  - **FR-01:** บันทึกออเดอร์ผ่านหน้าจอเลือกเมนูแยกตามหมวดหมู่
  - **FR-02:** บันทึกการชำระเงิน (เงินสด/QR) และออกใบเสร็จรับเงิน
  - **FR-03:** ตัดสต็อกอัตโนมัติและแจ้งเตือนเมื่อสต็อกต่ำกว่า 10 ชิ้น
  - **FR-04:** แสดงยอดขายแบบเรียลไทม์
  - **FR-05:** สร้างรายงานสรุปยอดขาย (Summary Sales Report)
  - **FR-09:** ระบบต้องอนุญาตให้แก้ไขหรือลบรายการในตะกร้าได้ก่อนยืนยัน
  - **FR-11 (Feedback จาก Prototype สัปดาห์ที่ 13):** ระบบต้องปิดการใช้งานปุ่ม '+ เพิ่ม' และแสดงสถานะ 'หมด' สำหรับเมนูที่ `stock_quantity = 0` เพื่อป้องกันข้อผิดพลาดตั้งแต่หน้าจอแรก
- **Non-Functional Requirements:** ความเร็วในการตอบสนองต่ำกว่า 1 วินาที, ความคงเส้นคงวา (Consistency) ของ Branch Selector ในทุกหน้าจอ, และความปลอดภัยของข้อมูล

---

4. User Stories และ Use Case Diagram

4.1 Use Case Diagram

ระบบประกอบด้วย 3 Actors หลัก:

1. **ลูกค้า (Customer):** สั่งสินค้า, ชำระเงิน
2. **พนักงานร้าน (Cashier/Barista):** รับออเดอร์, จัดการสต็อก, ออกใบเสร็จ
3. **เจ้าของร้าน (Owner):** ดูยอดขายแบบเรียลไทม์, สร้างรายงานยอดขาย, จัดการเมนู

4.2 ตัวอย่าง User Stories และ Acceptance Criteria

- **US-01:** *"ในฐานะพนักงานแคชเชียร์ ฉันต้องการเลือกเมนูเครื่องดื่มและระบุจำนวนผ่านหน้าจอ เพื่อให้ระบบคำนวณยอดเงินรวมให้โดยอัตโนมัติ"*
  - **Acceptance Criteria:**
    1. เมื่อคลิกเลือกเมนู รายการต้องไปปรากฏในตะกร้าพร้อมจำนวนเริ่มต้น 1
    2. ยอดรวมต้องอัปเดตทันทีเมื่อกดปุ่มเพิ่มหรือลดจำนวน
    3. ไม่สามารถระบุจำนวนสินค้าเกินกว่าสต็อกคงเหลือของสาขานั้นได้
- **US-02:** *"ในฐานะเจ้าของร้าน ฉันต้องการดูรายงานยอดขายรวมและเมนูขายดีตามช่วงเวลา เพื่อนำไปวางแผนสต็อกและโปรโมชั่น"*
  - **Acceptance Criteria:**
    1. รายงานต้องแสดงยอดเงินรวมและจำนวนออเดอร์ทั้งหมด
    2. จัดอันดับเมนูขายดีตามจำนวนแก้วที่ขายได้จริง

---

5. การออกแบบเชิงวัตถุ (OOAD)

5.1 Class Diagram & Object Diagram

ระบบออกแบบ Domain Model สอดคล้องกับหลักการ Object-Oriented Analysis and Design:

- **`Order` Class:** ประกอบด้วย `order_id`, `branch_id`, `employee_id`, `payment_method`, `created_at`
- **`OrderItem` Class:** ประกอบด้วย `order_item_id`, `order_id`, `menu_id`, `quantity`, `unit_price` และ method `getSubtotal()` สำหรับคำนวณราคาย่อย
- **`MenuItem` Class:** ประกอบด้วย `menu_id`, `branch_id`, `category_id`, `name`, `price`, `stock_quantity`

> ***หมายเหตุการออกแบบ 3NF:*** *คลาส `Order` ไม่เก็บฟิลด์ `total_amount` ในตารางฐานข้อมูล เนื่องจากเป็นค่าที่คำนวณได้ (Derived Attribute) จากผลรวมของ `OrderItem.getSubtotal()`*

---

6. การออกแบบฐานข้อมูล (Database Design)

6.1 ER Diagram และความสัมพันธ์ของตาราง

โครงสร้างฐานข้อมูลเชิงสัมพันธ์ประกอบด้วย 7 ตารางหลักที่เชื่อมโยงกันด้วย Foreign Key:

1. `branch` (1) ——< (M) `employee`
2. `branch` (1) ——< (M) `menu_item`
3. `category` (1) ——< (M) `menu_item`
4. `branch` (1) ——< (M) `orders`
5. `employee` (1) ——< (M) `orders`
6. `orders` (1) ——< (M) `order_item`
7. `menu_item` (1) ——< (M) `order_item`
8. `menu_item` (1) ——< (M) `stock_movement`

6.2 Data Schema (SQL DDL)

ระบบใช้ Schema ตามไฟล์ `wk07-schema.sql` พร้อมกำหนด Data Types, Constraints (NOT NULL, PRIMARY KEY, FOREIGN KEY, DEFAULT CURRENT_TIMESTAMP) ไว้อย่างรัดกุม

---

7. การออกแบบกระบวนการและพฤติกรรมระบบ

7.1 Sequence Diagram: กระบวนการสั่งซื้อและตัดสต็อก (Order to Stock)

กระบวนการยืนยันออเดอร์ (`POST /api/orders`) มีลำดับการทำงาน:

1. UI ส่ง `branchId`, `items`, `paymentMethod` ไปยัง Controller

2. Controller เริ่ม Transaction และล็อกแถวข้อมูลสต็อกด้วย `SELECT ... FOR UPDATE`
3. ตรวจสอบเงื่อนไขสต็อก หากไม่พอจะยกเลิก (Rollback) ทันที
4. บันทึกข้อมูลลงตาราง `orders` และ `order_item`
5. ตัดสต็อกใน `menu_item` และบันทึกประวัติใน `stock_movement`
6. ยืนยัน Transaction (Commit) และส่งผลตอบกลับให้ UI แสดงใบเสร็จ

7.2 Sequence Diagram: รายงานยอดขาย (`GET /api/reports/sales`)

เจ้าของร้านเลือกสาขาและช่วงเวลา → UI เรียก API → Controller ทำการ Query ข้อมูลยอดขายพร้อม Aggregate (`SUM` , `COUNT` , `GROUP BY`) → ส่งข้อมูล JSON สรุปผลกลับมาแสดงผลเป็นการ์ด KPI และตารางสรุป

---

8. การออกแบบ Input/Output และส่วนประสานผู้ใช้

8.1 การออกแบบ Input Form (หน้าจอ POS)

- มี **Branch Selector** อยู่ตำแหน่งเดียวกันทุกหน้าจอ (Consistency)
- การ์ดเลือกเมนูแสดงรูปภาพ ราคา และจำนวนสต็อกคงเหลือ
- ป้ายแจ้งเตือนสีส้มสำหรับสต็อกต่ำ (< 10) และป้ายสีเทาปิดการใช้งานสำหรับสต็อกหมด (= 0)
- ตารางตะกร้าสินค้าพร้อมปุ่มปรับจำนวน [- 1 +] และปุ่มลบรายการ

8.2 การออกแบบ Output (ใบเสร็จรับเงิน & หน้ารายงาน)

- **ใบเสร็จรับเงิน (Receipt):** แสดงชื่อร้าน Minions Cafe, สาขา, เลขที่ออเดอร์, วันเวลา, รายการสินค้าพร้อมราคาต่อหน่วย, ยอดรวมทั้งสิ้น, วิธีชำระเงิน, และชื่อพนักงานแคชเชียร์
- **รายงานยอดขาย (Sales Report):** สรุปยอดขายรวม, จำนวนออเดอร์, เมนูขายดีอันดับ 1, ตารางแจกแจงยอดขายต่อเมนู, และกราฟเปรียบเทียบระหว่างสาขา

---

9. สถาปัตยกรรมระบบ (4+1 View Model)

9.1 Physical View

ระบบรันในรูปแบบ Single Development Machine Deployment (`localhost`):

- Client Browser เชื่อมต่อผ่าน HTTP พอร์ต `3000`
- Application Server รัน Node.js Monolith Process
- Database Server รัน MySQL Server daemon พอร์ต `3306`

9.2 Development View

จัดโครงสร้างแบบ Layered Architecture ภายใน Monolith:

- `Presentation Layer` : หน้าเว็บ HTML, CSS, JavaScript (Interactive Frontend)
- `Routes Layer` : กำหนด RESTful Endpoints (`/api/menu` , `/api/orders` , `/api/reports/sales`)

- `Controllers Layer` : ควบคุม Business Logic และ Transaction
- `Models Layer` : Domain Entities (`Order` , `OrderItem` , `MenuItem`)
- `Config Layer` : การจัดการ Connection Pool (`mysql2/promise`)

---

10. ผลการทดสอบ Usability

จากการทดสอบ Task-Based Testing กับผู้ใช้งานจริงในสัปดาห์ที่ 12:

- **สิ่งที่ค้นพบ:** ผู้ใช้ใหม่สามารถสั่งซื้อสินค้าได้สำเร็จภายใน 42 วินาที แต่พบปัญหา Critical คือเมนูที่สต็อกเป็น 0 ยังกดเลือกได้ ทำให้เกิดข้อผิดพลาดภายหลัง และปัญหา Major คือปุ่มยืนยันออเดอร์ไม่โดดเด่น
- **การแก้ไข:** ปรับปรุง UI ให้ปิดปุ่ม (Disable) เมื่อสต็อกเป็น 0 พร้อมขึ้นป้าย "สินค้าหมด" และเปลี่ยนปุ่มยืนยันออเดอร์ให้เป็นสีเขียวเด่นชัด (Emerald Accent) พร้อมเพิ่มระบบคำนวณเงินทอนอัตโนมัติ

---

11. ผลการรวมระบบ (System Integration)

ในสัปดาห์ที่ 13 ได้ทำการรวมโมดูลทั้งหมด (Order, Menu, Stock, Report) เข้าด้วยกัน และทำการทดสอบแบบ End-to-End ครบทุก Happy Path และ Error Path:

1. การเลือกสินค้าหลายสาขาถูกต้องตาม `branch_id`
2. การคำนวณราคารวมอัตโนมัติ
3. การยืนยันออเดอร์และตัดสต็อกสินค้าในระบบจริง
4. การเปิดหน้ารายงานยอดขายพบว่าออเดอร์ใหม่ปรากฏในยอดขายทันทีโดยไม่ต้องรีโหลด

---

12. ผลการตรวจสอบคุณภาพและความปลอดภัย

จากการตรวจสอบโค้ดจริงตาม Code Review Checklist ในสัปดาห์ที่ 14:

- ตรวจพบความเสี่ยงระดับ **Critical** เรื่อง **Race Condition** ในการอ่านและตัดสต็อกเมื่อมีคำสั่งซื้อพร้อมกัน
- **การแก้ไข:** ปรับปรุงโค้ดด้วยการใช้ **Database Transaction** และ `SELECT ... FOR UPDATE` เพื่อล็อกแถวข้อมูลสต็อก ป้องกันสต็อกติดลบได้อย่างเด็ดขาด
- ตรวจพบความเสี่ยงระดับ **High** เรื่อง IDOR ในการแก้ไขเมนู และได้วางแผนแนวทางตรวจสอบสิทธิ์ควบคู่กับ `branch_id`

---

13. สรุปผลและข้อเสนอแนะ (Conclusions & Recommendations)

13.1 บทเรียนที่ได้รับ (Lessons Learned)

- การออกแบบระบบที่ดีต้องเชื่อมโยงสอดคล้องกันเป็นกระบวนการเดียว (Traceability) ตั้งแต่ Requirement, Diagram ไปจนถึง Source Code จริง
- การทดสอบกับผู้ใช้จริง (Usability Testing) มีความสำคัญอย่างยิ่งในการค้นหาข้อบกพร่องที่ทีมพัฒนาอาจมองข้าม
- ระบบที่ทำงานได้ (Functional) ต้องควบคู่ไปกับความปลอดภัยและความถูกต้องของข้อมูล (Security & Data Integrity)

13.2 ข้อจำกัดและแนวทางพัฒนาต่อยอด (Future Improvements)

- พัฒนาระบบสมาชิกและสะสมแต้มให้สมบูรณ์ (Loyalty Program)
- เชื่อมต่อ Payment Gateway สำหรับสแกน QR Code แบบ Dynamic PromptPay อัตโนมัติ
- แยกบริการบางส่วนเป็น Microservices เมื่อขยายสาขามากกว่า 20 สาขา

---

14. ภาคผนวก (AI Disclosure Forms)

เอกสาร AI Disclosure ทุกฉบับตลอดภาคการศึกษาได้รับการจัดเก็บไว้ในโฟลเดอร์ `docs/ai-disclosure/` :

- AI Disclosure - สัปดาห์ที่ 5
- AI Disclosure - สัปดาห์ที่ 9
- AI Disclosure - สัปดาห์ที่ 10
- AI Disclosure - สัปดาห์ที่ 13